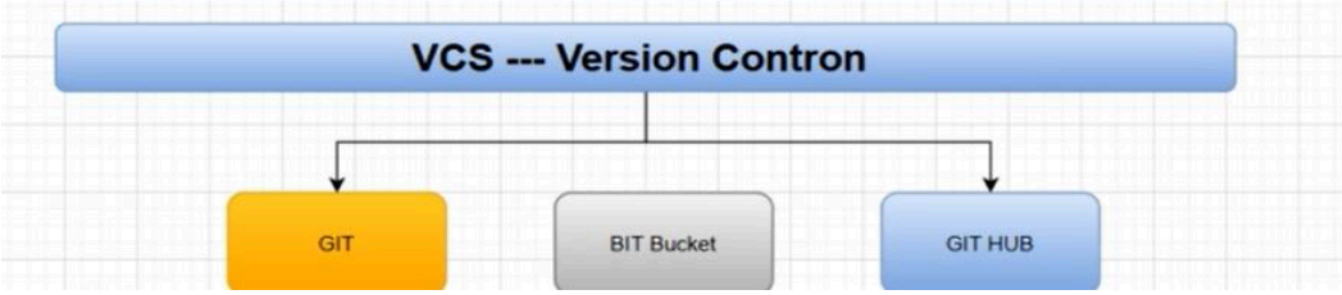


A VCS (Version Control System) is essential for tracking Code history, enabling collaboration, providing backup, supporting experimentation, ensuring accountability and documentation, and automating deployment — all for the safe management of projects, especially in software development.

For example –

- **Tracking:** if you accidentally delete a function in your code, VCS lets you restore the last working version.
- **Collaboration:** Multiple people can work on the same project without overwriting each other's work by using individual branches. After changes are done, VCS merges their work safely.
- **Backup:** If your computer crashes, the remote repository (e.g., GitHub) has a backup of your project.
- **Experimentation:** VCS allows you to create branches to try new features or ideas without affecting the main project. If the experiment fails, you can simply delete the branch.
- **Accountability & Documentation:** Each commit is like a timestamped note explaining changes. This helps team members and future developers understand the project's history.
- **Automation & Deployment:** VCS integrates with CI/CD pipelines to automatically test and deploy code, ensuring the project is always working and up to date in production.

So,

- **Git** is a type of **Version Control System (VCS)**, specifically a **Distributed Version Control System (DVCS)**.
  - It tracks changes, manages branches, and lets multiple people work on a project safely.
- **GitHub** is a **platform/service that hosts Git repositories online**.
  - It provides collaboration tools and additional features that make working with Git easier.

GitHub features include:

- Collaboration (pull requests, code reviews)
- Backup of code in the cloud
- Issue tracking and project management

1. What is Git?

- Git is a version control system.
- It helps to track changes in your **code or files** over time.
- Think of it like a "save button" on steroids: you can go back to any previous version of your project if something breaks.
- You can also work with others on the same project without messing up each other's code.

Example:

Imagine you're writing a book:

- You write Chapter 1 → Git saves it.
- You add Chapter 2 → Git saves it.
- Later you don't like Chapter 2 → Git lets you go back to the version with just Chapter 1.

2. What is GitHub?

- GitHub is an online platform that hosts Git repositories.
- It's like **Google Drive for your code** which is made for developers.
- You can share your code with others, collaborate, and even show your work to the world.

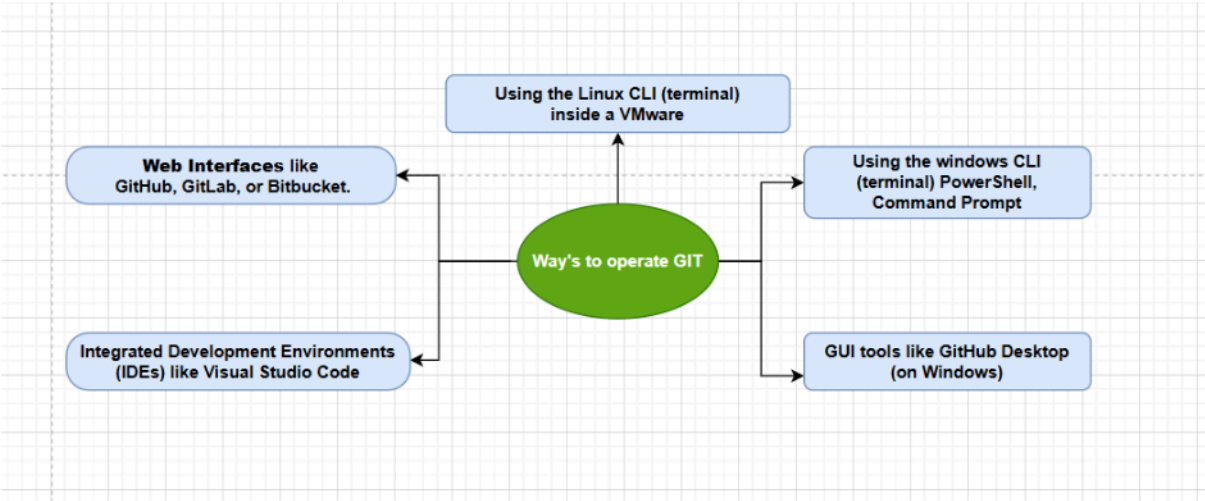
Example:

- You create a project on your computer using Git & upload it to GitHub.
- Your friend can download it, make changes, and send it back to you.

3. Why we use Git and GitHub?

- Track changes: Never lose your work. Go back to old versions easily.
- Collaboration: Multiple people can work on the same project safely.
- Backup: Your code is safe online on GitHub.
- Portfolio: You can showcase your projects to employers.
- Open-source contribution: You can contribute to other people's projects.

We can use them in different ways:



1. **Linux CLI (VMware/WSL):** full control in a Linux environment.
2. **Windows CLI (PowerShell/CMD):** quick commands in native Windows.
3. **GUI tools (GitHub Desktop):** easy visual commits & branches.
4. **Web interfaces (GitHub/GitLab/Bitbucket):** manage repos & pull requests online.
5. **IDEs (VS Code):** code, commit & push without leaving the editor.

1. Git Workflow

Workflows define **how developers collaborate**, manage branches, and merge code.
A **Git workflow** is basically **how developers organize their Git usage and collaboration**. It defines the **process of how code moves from development to production**.
detailed.

1. Repository Setup

- **Central repository:** Usually on GitHub, GitLab, or Bitbucket. This is the main "official" version of the project.
- **Local clone:** Each developer clones the repository to their computer to work locally.

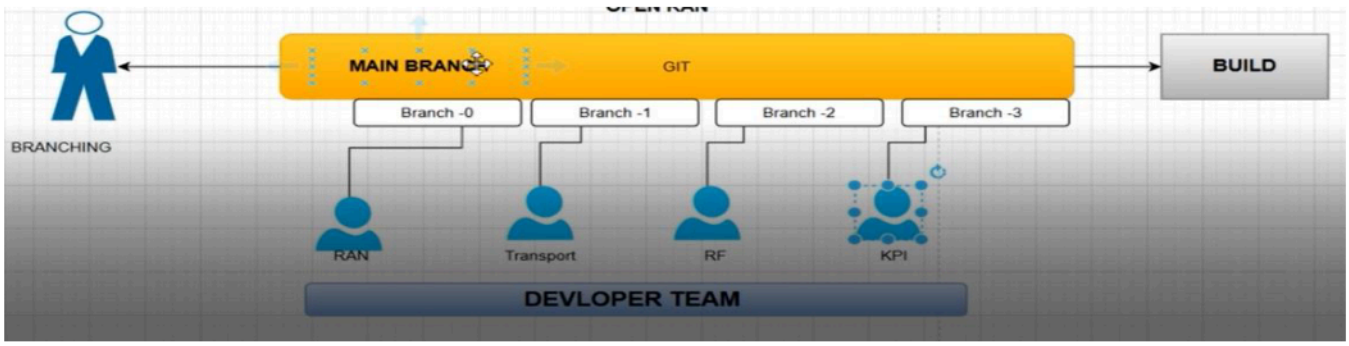
Bash Command example:

```
git clone https://github.com/user/project.git
```

2. Branching Strategy

In large projects, branching is key to avoid conflicts like :

- **Main branch (master)** – Stable version of the project; always deployable.
- **Developer branch (develop)** – Integration branch where features are combined before going to main.
- **Feature branches (feature/xyz)** – Separate branches for each new feature.
- **Bugfix branches (bugfix/xyz)** – Fix specific issues.
- **Release branches (release/v1.0)** – Prepare for a release.



Bash Command example:

```
git checkout -b feature/login-system
```

3. Making Changes

1. Work on your **local branch**.
2. **Add changes** to staging area:
   `git add`
3. **Commit changes** with a descriptive message:
   `git commit -m "Add login form validation"`

Best practice: Commit often, with clear messages explaining the changes.

4. Keeping Your Branch Updated

In large projects, other developers are also pushing changes. You need to stay updated:
use `git pull` (fetch + merge).

5. Code Review and Pull Requests

- Once your feature is ready, push your branch to the remote repository:
  `git push origin feature/login-system`
- Create a **Pull Request (PR)** on GitHub (or Merge Request on GitLab).
- Other team members **review your code**, suggest changes, or approve it.

6. Merging Changes

- After approval, your feature branch is merged into **develop** or **main**.
- Usually done with a **merge commit** or **rebase** to keep history clean.
- Conflicts (if two people changed the same code) are resolved manually.

7. Testing (Git pipeline)

- Large projects usually have **CI/CD pipelines** (Continuous Integration/Deployment) to automatically test the code after merging.
- Ensures your changes don't break the project.

A **Git pipeline** is a bit different — it usually refers to **automating steps after code is pushed to a repository**, often as part of **CI/CD (Continuous Integration / Continuous Deployment)**.

Example of a Git pipeline

1. **Developer commits and pushes code** to a branch.
2. **CI server** (like GitHub Actions, GitLab CI, Jenkins) automatically:
   - Runs **tests**
   - Performs **code quality checks**
   - Builds the project (if needed)
3. **If all checks pass**, the code may be:
   - Merged into main/develop branch
   - Deployed to staging or production environment

8. Releasing

- When ready, the **develop branch** is merged into **main** via a release branch.
- Version tags are added:

```
git tag -a v1.0 -m "Release version 1.0"
git push origin v1.0
```

- Then deployed to production.

9. Hotfixes if Needed

- For urgent bugs in production, create a **hotfix branch** from main:

```
git checkout -b hotfix/fix-login main
```

- Fix, test, and merge back into **main** and **develop.**

Summary of Git Workflow in Large Projects

1. Clone the repo.
2. Create a branch (feature/bugfix).
3. Make changes → commit → push.
4. Create Pull Request → code review.
5. Merge into develop/main.
6. Test with CI/CD.
7. Release to production.
8. Hotfix if needed.

key features of Git and GitHUB and workflows:

◇ **Git** is a **distributed version control system** that helps developers track changes in code, collaborate efficiently, and manage project history.

1. **Clone** – First, make a copy of the repository (means copy a folder) from the remote server (means from GitHub) to your local computer.
2. **Branch** – Create a separate branch to work on a new feature or fix without affecting the main code.
3. **Commit** – Save snapshots of your changes locally as you work.
4. **Pull** – Before sharing, fetch and integrate any changes from the remote repository to avoid conflicts.
5. **Push** – Upload your local commits to the remote repository so others can see your work.
6. **Merge** – Combine changes from different branches into one, usually after a pull request or review.

◈ **GitHub** is a **cloud-based hosting platform for Git repositories** that adds powerful collaboration features:

- Centralized place for sharing code.
- **Pull requests & code reviews** to ensure quality before merging.
- **Issues & discussions** for project management.
- **Actions (CI/CD)** to automate testing and deployment.
- Easy integration with teams and open-source communities.

☞ Together, Git and GitHub make it easy to build, collaborate, and scale software projects.

Git & GitHub Glossary

General Git Terms

- **Repository (Repo)** → A project folder tracked by Git that stores code and its entire history.
- **Commit** → A saved "snapshot" of your project at a certain point in time. "Snapshot" means Git takes a picture of your project's files at that exact moment in time. This snapshot includes **all your code, structure, and changes.**
- **Branch** → A separate line of development used to work on features or fixes without affecting the main code.
- **Merge** → Combining changes from one branch into another.
- **Clone** → Making a local copy of a repository from a remote server (like GitHub).
- **Pull** → Fetching changes from the remote repo and merging them into your local repo.
- **Push** → Uploading your local commits to a remote repo (like GitHub).
- **Remote** → The online version of your repo (e.g., GitHub) that your local repo connects to.
- **HEAD** → A pointer to the latest commit in your current branch.
- **Checkout** → Switching to another branch or commit.
- **Staging Area (Index)** → A temporary area where you prepare changes before committing.
- **Diff** → Shows the difference between two versions of a file or commit.

GitHub-Specific Terms

- **Fork** → A personal copy of someone else's repo on GitHub that you can edit freely.
- **Pull Request (PR)** → A request to merge your changes from your branch (or fork) into another branch/repo. Often used for code review.
- **Issue** → A way to track bugs, tasks, or feature requests in a GitHub project.
- **Star** → A way to bookmark or show appreciation for a project on GitHub.
- **Watch** → Subscribe to notifications about a repo's activity.
- **Action (GitHub Actions)** → Automation tool for CI/CD (testing, deployment, etc.) in GitHub.
- **Release** → A packaged version of the code (with a version number) ready for use.

Other Useful Terms

- **Tag** → A label for specific commits (often used for marking releases, like v1.0).
- **Conflict** → Happens when Git can't automatically merge changes because the same part of a file was edited differently.
- **Rebase** → Reapplying commits on top of another base branch (used to clean history).
- **Cherry-pick** → Applying a specific commit from one branch to another.
- **Origin** → The default name for the remote repo when you first clone it.

Note – This is a summarized explanation for the basic understanding of Git and its workflow.

Surya Prakash